

Пример 10. Адаптер (Adapter).

```
# include <iostream>
# include <memory>

using namespace std;

class Adapter
{
public:
    virtual ~Adapter() = 0;

    virtual void request() = 0;
};

Adapter::~Adapter() = default;

class BaseAdaptee
{
public:
    virtual ~BaseAdaptee() = 0;

    virtual void specificRequest() = 0;
};

BaseAdaptee::~BaseAdaptee() = default;

class ConAdapter : public Adapter
{
private:
    shared_ptr<BaseAdaptee> adaptee;

public:
    ConAdapter(shared_ptr<BaseAdaptee> ad) : adaptee(ad) {}

    virtual void request() override;
};

class ConAdaptee : public BaseAdaptee
{
public:
    virtual void specificRequest() override { cout << "Method ConAdaptee;" << endl; }
};

#pragma region Methods
void ConAdapter::request()
{
    cout << "Adapter: ";

    if (adaptee)
    {
        adaptee->specificRequest();
    }
    else
    {
        cout << "Empty!" << endl;
    }
}

#pragma endregion

int main()
{
    shared_ptr<BaseAdaptee> adaptee(new ConAdaptee());
    shared_ptr<Adapter> adapter(new ConAdapter(adaptee));

    adapter->request();
}
```

Пример 11. Декоратор (Decorator).

```

#include <iostream>
#include <memory>

using namespace std;

class Component
{
public:
    virtual ~Component() = 0;

    virtual void operation() = 0;
};

Component::~~Component() {}

class ConComponent : public Component
{
public:
    virtual void operation() override { cout << "ConComponent "; }
};

class Decorator : public Component
{
protected:
    shared_ptr<Component> component;

public:
    Decorator(shared_ptr<Component> comp) : component(comp) {}
};

class ConDecorator : public Decorator
{
public:
    using Decorator::Decorator;

    virtual void operation() override;
};

#pragma region Method
void ConDecorator::operation()
{
    if (component)
    {
        component->operation();

        cout << "ConDecorator ";
    }
}

#pragma endregion

int main()
{
    shared_ptr<Component> component(new ConComponent());
    shared_ptr<Component> decorator1(new ConDecorator(component));

    decorator1->operation();
    cout << ";" << endl;

    shared_ptr<Component> decorator2(new ConDecorator(decorator1));

    decorator2->operation();
    cout << ";" << endl;
}

```

Пример 12. Компоновщик (Composite).

```

#include <iostream>
#include <memory>
#include <vector>
#include <iterator>

```

```

using namespace std;

class Component;

using VectorComponent = vector<shared_ptr<Component>>;
using IteratorComponent = VectorComponent::const_iterator;

class Component
{
public:
    virtual ~Component() = 0;

    virtual void operation() = 0;

    virtual bool isComposite() const { return false; }
    virtual bool add(shared_ptr<Component> comp) { return false; }
    virtual bool remove(const IteratorComponent& it) { return false; }
    virtual IteratorComponent begin() const { return IteratorComponent(); }
    virtual IteratorComponent end() const { return IteratorComponent(); }
};

Component::~~Component() {}

class Figure : public Component
{
public:
    virtual void operation() override { cout << "Figure method;" << endl; }
};

class Camera : public Component
{
public:
    virtual void operation() override { cout << "Camera method;" << endl; }
};

class Composite : public Component
{
private:
    VectorComponent vec;

public:
    Composite() = default;
    Composite(shared_ptr<Component> first, ...);

    virtual void operation() override;

    virtual bool isComposite() const override { return true; }
    virtual bool add(shared_ptr<Component> comp) { vec.push_back(comp); return false; }
    virtual bool remove(const IteratorComponent& it) { vec.erase(it); return false; }
    virtual IteratorComponent begin() const override { return vec.begin(); }
    virtual IteratorComponent end() const override { return vec.end(); }
};

#pragma region Methods
Composite::Composite(shared_ptr<Component> first, ...)
{
    for (shared_ptr<Component>* ptr = &first; *ptr; ++ptr)
        vec.push_back(*ptr);
}

void Composite::operation()
{
    cout << "Composite method;" << endl;
    for (auto elem : vec)
        elem->operation();
}

#pragma endregion

int main()

```

```

{
    using Default = shared_ptr<Component>;
    shared_ptr<Component> fig1(new Figure()), fig2(new Figure()), cam1(new Camera()), cam2(new Camera());
    shared_ptr<Component> composite1(new Composite(fig1, cam1, fig2, cam2, Default()));

    composite1->operation();
    cout << endl;

    IteratorComponent it = composite1->begin();

    composite1->remove(++it);
    composite1->operation();
    cout << endl;

    shared_ptr<Component> composite2(new Composite(shared_ptr<Component>(new Figure()), composite1, Default()));

    composite2->operation();
}

```

Пример 13. Заместитель (Проксу).

```

#include <iostream>
#include <memory>
#include <map>
#include <random>

using namespace std;

class Subject
{
public:
    virtual ~Subject() = 0;

    virtual pair<bool, double> request(size_t index) = 0;
    virtual bool changed() { return true; }
};

Subject::~~Subject() = default;

class RealSubject : public Subject
{
private:
    bool flag{ false };
    size_t counter{ 0 };

public:
    virtual pair<bool, double> request(size_t index) override;
    virtual bool changed() override;
};

class Proxy : public Subject
{
protected:
    shared_ptr<RealSubject> realsubject;

public:
    Proxy(shared_ptr<RealSubject> real) : realsubject(real) {}
};

class ConProxy : public Proxy
{
private:
    map<size_t, double> cache;

public:
    using Proxy::Proxy;

    virtual pair<bool, double> request(size_t index) override;
};

#pragma region Methods
bool RealSubject::changed()

```

```

{
    if (counter == 0)
    {
        flag = true;
    }
    if (++counter == 7)
    {
        counter = 0;
        flag = false;
    }
    return flag;
}

pair<bool, double> RealSubject::request(size_t index)
{
    random_device rd;
    mt19937 gen(rd());

    return pair<bool, double>(true, generate_canonical<double, 10>(gen));
}

pair<bool, double> ConProxy::request(size_t index)
{
    pair<bool, double> result;

    if (!realsubject)
    {
        cache.clear();

        result = pair<bool, double>(false, 0.);
    }
    if (!realsubject->changed())
    {
        cache.clear();

        result = realsubject->request(index);

        cache.insert(map<size_t, double>::value_type(index, result.second));
    }
    else
    {
        map<size_t, double>::const_iterator it = cache.find(index);

        if (it != cache.end())
        {
            result = pair<bool, double>(true, it->second);
        }
        else
        {
            result = realsubject->request(index);

            cache.insert(map<size_t, double>::value_type(index, result.second));
        }
    }

    return result;
}

#pragma endregion

int main()
{
    shared_ptr<RealSubject> subject(new RealSubject());
    shared_ptr<Subject> proxy(new ConProxy(subject));

    for (size_t i = 0; i < 21; ++i)
    {
        cout << "(" << i + 1 << ", " << proxy->request(i % 3).second << ")" << endl;

        if ((i + 1) % 3 == 0)
            cout << endl;
    }
}

```

```
}
```

```
}
```

Пример 14. Мост (Bridge).

```
# include <iostream>
```

```
# include <memory>
```

```
using namespace std;
```

```
class Implementor
```

```
{
```

```
public:
```

```
    virtual ~Implementor() = 0;
```

```
    virtual void operationImp() = 0;
```

```
};
```

```
Implementor::~~Implementor() = default;
```

```
class Abstraction
```

```
{
```

```
protected:
```

```
    shared_ptr<Implementor> implementor;
```

```
public:
```

```
    Abstraction(shared_ptr<Implementor> imp) : implementor(imp) {}
```

```
    virtual ~Abstraction() = 0;
```

```
    virtual void operation() = 0;
```

```
};
```

```
Abstraction::~~Abstraction() = default;
```

```
class ConImplementor : public Implementor
```

```
{
```

```
public:
```

```
    virtual void operationImp() override { cout << "Implementor;" << endl; }
```

```
};
```

```
class Entity : public Abstraction
```

```
{
```

```
public:
```

```
    using Abstraction::Abstraction;
```

```
    virtual void operation() override { cout << "Entity: "; implementor->operationImp(); }
```

```
};
```

```
int main()
```

```
{
```

```
    shared_ptr<Implementor> implementor(new ConImplementor());
```

```
    shared_ptr<Abstraction> abstraction(new Entity(implementor));
```

```
    abstraction->operation();
```

```
}
```